Upstream: what YARG's own rules say, and what we are asking them

The project's aim is that work here be upstreamable rather than a fork that diverges forever. This document records what upstream's process actually is — read from their documentation rather than assumed — and what we have asked them.

Their process, from `CONTRIBUTING.md`

- **Ask on Discord before building a feature.** Their words: "*It's recommended that you ask there before working on a new feature, in case someone is already working on a feature/change.*" Discord: https://discord.gg/sqpu4R552r. Task tracker: https://yarg.youtrack.cloud/agiles/147-7/current.
- **PRs target `dev`. A PR based on `master` will not be accepted.**
- Every feature falls in one of six tiers, and the tier decides whether a PR is even looked at:

Tier	What it means for a PR
In-Development	Do not PR without coordination — they are already on it
Planned	PRs welcome
Stretch Goal	Discouraged, but experimenting and sharing progress is invited, and "*large progress on a stretch goal may promote it to a higher tier*"
Eligible	PRs welcome; they will not build it themselves
Problematic	Heavily discouraged; big headachey impacts elsewhere
Out of Scope	"*Do NOT PR these features. Your PR will immediately be denied*"

Their Out of Scope examples include CON Decryption. That is worth noting: this project independently made CON/mogg decryption a permanent non-goal on legal grounds, and upstream has independently ruled it out on their own. Two of our three permanent non-goals are

things upstream would reject on sight. We are not asking them for anything they have already refused.

Which tier a remote song source falls into is not published, and none of the tier examples resemble it. That is the first question to ask, and it is exactly the question their contributing guide tells contributors to ask.

What we searched before asking

Nothing found in their issues proposes loading songs from a remote server. The nearest is **#860, "[FR] Built in Web Server for Song Queing/Search from external device"** — a *control plane* (search and queue from a phone) rather than a content source. Worth citing so it is clear we looked, and worth not conflating with what we are proposing.

But it is not merely adjacent, and that is worth saying in the post. #860 has been open since August 2024, unlabelled, with a comment pointing at a second Discord proposal that adds up/down votes on the queue — so there is demand and nobody has built it. The reason nobody has is in the request's own words: "*whilst YARG is running*". Nothing outside the game can reach a running client. **A remote song source is the thing that could** — the same channel that fetches songs can carry a queue. So this proposal is not novel work competing for their attention; it is the missing half of a request they already have.

Also searched and not found: any issue or PR about shared libraries, syncing songs between machines, or a network song source. `#1030 "Add support for user-supplied song sources"` is about **source icons**, not sources of songs.

The shape of the ask, and why it is smaller than it sounds

The server already hands out **plain `.sng` files that unmodified YARG reads natively** — that is the design commitment in ADR-001, and it has been demonstrated on two machines, two operating systems and two CPU architectures. So we are not asking upstream to support a protocol in order to play our songs. They already can.

What a player cannot do today is get those songs **without running a separate sync tool**. So the upstream ask is narrow: a way for YARG to discover and fetch songs from a URL.

The ask got smaller once the code was read

The paragraph that used to sit here said the seam was that `SongEntry` is abstract while `ActualLocation`, `SortBasedLocation` and `GetLastWriteTime()` assume a local path. True, and not useful — it names a symptom, and anyone who maintains YARG.Core would know that already. `ADR-004` replaced it with what the code actually says, and the ask that falls out is much smaller and much more concrete:

- **`SngFile` has exactly one loader**, `TryLoadFromFile(string, bool)` (`IO/SngHandler/SngFile.cs:100`), and it opens a `FileStream` itself. There is no stream-taking overload — but the method is **already stream-oriented after its first handful of lines**: it wraps the file in either `YARGSongFileStream` or the raw `FileStream`, assigns `tracker.Stream`, and everything after that reads only from that stream. So `TryLoadFromStream` is an extract-method, not a redesign. `FixedArray` already reads from streams (`ReadRemainder(Stream)`, `Read(Stream, long, bool)`).
- **`SngEntry`, `UnpackedIniEntry` and their base `IniSubEntry` are all `internal`** with private constructors, so nobody outside the assembly can add an entry type. Anything at the entry level has to happen inside YARG.Core, by them or with them.
- **One source seam already exists and does not go far enough**: `protected abstract FixedArray<byte>? GetChartData(string filename)` (`SongEntry.IniBase.cs:82`). `LoadChart()` is genuinely source-agnostic on top of it; everything else is not — `SngEntry` reaches for `SngFile.TryLoadFromFile(_location, …)` at **seven** sites and the local filesystem at nine more.
- **YARG.Core contains no networking at all** — zero matches for `HttpClient|System.Net|UnityWebRequest` across the library. So the right place for a fetch is the Unity layer, which already networks, and **we are not asking them to put an `HttpClient` in YARG.Core.** Saying that out loud is worth a sentence, because it is the objection a maintainer would reach for first.

So the post below asks for **two small things** rather than for a feature: `TryLoadFromStream`, which is useful on its own and has no remote-library baggage, and — separately, lower confidence — whether a materialise-before-load hook on `IniSubEntry` is the kind of thing they would ever want.

Everything above was measured against `yarg` `3673672` / `YARG.Core` `028969a` on 2026-09-07, and should be re-checked before the post goes out if that is much later. Citing a line number that has moved is a bad first impression in a channel full of people who know the file.

Draft: the Discord post

Not yet sent. Jay sends it, under his own account; nothing goes out without him.

Hey folks — I've been building a self-hosted song server for YARG and wanted to ask here before going further, per CONTRIBUTING.

The short version: a small Go server that indexes a song folder and hands out plain `.sng` files. It is not a game modification — **unmodified YARG already reads everything it serves**, and I've had that running across two machines, two OSes and two CPU architectures. Today a little sync client pulls songs into a folder and YARG scans it like any other folder. The obvious next step is YARG pointing at a server URL directly, instead of needing a separate tool.

I read through `YARG.Core` and the Unity side before writing this, so I can ask something more specific than "would you like this feature". Five questions, smallest first — plus one thing I ran into that may be a bug rather than a request:

1. Would you take `SngFile.TryLoadFromStream(Stream, bool)` on its own? `TryLoadFromFile` opens the `FileStream` itself, but everything after the first few lines already works purely off `tracker.Stream` — so this looks like an extract-method rather than a redesign, and `FixedArray` already has stream readers. It's useful for anything that has `.sng` bytes without a path on disk, remote or not. If that's welcome I'd happily open that PR by itself and leave everything below for later.

2. Is a "materialise before load" hook on `IniSubEntry` something you'd ever want? Lower confidence on this one. `GetChartData` is already an abstract seam and `LoadChart` is source-agnostic on top of it, but the other loaders go through `SngFile.TryLoadFromFile(_location, …)` — seven call sites in `SngEntry` — so a one-line hook the load methods call first, defaulting to a no-op, seems less invasive than abstracting `_location`. Very open to being told that's the wrong shape.

3. Would a second "delivered out of band" song folder be objectionable in principle? `SongContainer.RunRefresh` already appends `PathHelper.SetlistPath` to the scan list — a folder the player didn't add, populated by the Launcher, skipped cleanly when absent. What I'd be adding is a second producer for that same shape of folder. If that pattern is fine, most of this needs nothing from `YARG.Core` at all.

4. What should a release build do about plain HTTP? `insecureHttpOption` is `NotAllowed`, and a song server on someone's LAN is plain HTTP — so the feature simply doesn't work in a release build as things stand. I've set it to `DevelopmentOnly` in my fork to get the thing testable, which ships nothing weaker to players, but that's postponing the question rather than answering it. Relax it, require HTTPS from the server, or make it an opt-in per host? I'd rather match whatever you'd want than pick one and find out later.

5. Is `dev` meant to refuse a song whose `song.ini` has no `song_length`? Not part of the above — I hit it while testing and it looks like it might be a regression, so I'd rather mention it than not. Same song both times, chart copied byte for byte, 3 MB ogg, the two folders differing by exactly one line of `song.ini`:

- v0.15.0: both accepted, no `badsongs.txt` at all
- `dev` (`3673672`): the one without `song_length` refused, "*Corruption of either the ini file or chart/mid file*"

If that's deliberate, no problem. Two things in case they help: the message points at the chart, which sent me looking in the wrong place for a while — and I checked before assuming this was a big deal, so I can say it isn't: of 256 real community charts I have here, **all 256 declare `song_length`**, so nobody's library is breaking. It only showed up because my own synthetic test corpus writes minimal `song.ini` files. Happy to open an issue with the two folders if it's worth having.

6. Which tier does a remote song source fall into, and is anyone already on it? It doesn't match any of the CONTRIBUTING examples and I couldn't find an issue for it — the closest is #860, which is search/queue from a phone rather than a source of songs. Happy to stay out of the way if someone's already working on it.

To be explicit about what I'm *not* asking for: **no `HttpClient` in `YARG.Core`.** There's no networking in the library at all right now and I don't think this needs to change that — the fetching belongs in the Unity layer, which already networks. Also no CON decryption, no touching `songcache.bin`, and no distributing copyrighted audio. The server serves what the operator already has.

I'm building it in my fork either way, so this isn't a request for anyone to do work. I'd just rather build it in a shape you'd consider than find out later it was never going to fit.

One thing in case it makes this more interesting rather than less: the same channel would carry a **queue**. #860 has been open since 2024 asking for search-and-queue from a phone while YARG is running, and as far as I can tell it's unbuilt because nothing outside the game can reach a running client. A remote source is the thing that could. Not proposing that part now — just noting the two are the same plumbing.

Everything's LGPL-3.0-or-later, same as YARG: https://github.com/Coffeehedake/yarg-song-server

One rule about that link

The post links the public GitHub mirror, never `gitlab.badassium.com`. Origin is a GitLab instance on a home server; GitHub is a downstream, force-overwritten mirror that exists precisely so there is something public to point at. Pasting the origin URL into a Discord with a few thousand strangers in it would advertise home infrastructure for no benefit — the mirror carries identical content.

Verified before this draft went anywhere: `Coffeehedake/yarg-song-server` is public, LGPL-3.0, and its last push matches the last push to origin, so the mirror is live rather than a stale snapshot from months ago.

What to do with the answer

Question 1 is the one that matters most, and it is deliberately separable. A yes to `TryLoadFromStream` is a merged PR and a working relationship with upstream regardless of what happens to the rest; a no to everything else costs nothing we were counting on, because `ADR-004` increment 1 needs no YARG.Core change at all.

- **In-Development or someone is on it** — stop, and offer what we have to whoever is.
- **Planned or Eligible** — design toward a PR from the start, and target `dev`.
- **Stretch Goal** — build it in the fork and share progress; their own rule says large progress can promote a tier.
- **Problematic or Out of Scope** — build it in the fork for ourselves, drop the upstreaming goal for this feature specifically, and say so plainly in the ROADMAP rather than leaving a dead aspiration in it.

Their answer does not block Phase 3. It shapes it.
